Inventory Management System — Project Documentation

Project Name: Inventory System ITI
Version: 1.0
Stack: Vanilla JavaScript (ES Modules), jQuery, Bootstrap 5, JSON Server
GitHub Repository: https://github.com/Renad-Elgabrouny/Inventory-System

---

Team Members

Renad Elgabrouny

Fatma Mekawy

Mohamed Attia

Karim Yasser

Table of Contents

1. Project Overview
2. Folder Structure
3. API Endpoints
4. Pages & Features
5. Services
6. Models & Validation
7. Error Handling
8. How to Run the Project

---

1. Project Overview

The Inventory Management System is a front-end web application built for managing warehouse inventory. It provides real-time tracking of products, suppliers, purchase orders, users, and activity logs. The application communicates with a local **JSON Server** REST API that persists data in a `db/products.json` file.

Key Capabilities:

- Role-based access control (Admin vs. User)
- Full CRUD for products, suppliers, users, and orders
- Stock adjustment with audit trail
- Low-stock alerts and inventory value reporting
- Activity logging for every significant operation

- Authentication via localStorage session

---

2. Folder Structure

```
Inventory-System/
│
├── index.html                    # Main dashboard (SPA shell)
├── login.html                    # Login page
├── register.html                 # Registration page
│
├── pages/                        # Feature page fragments (loaded into
dashboard)
│   ├── orders.html               # Purchase Orders page
│   ├── products.html             # Products management page
│   ├── reports.html              # Inventory reports page
│   ├── suppliers.html            # Suppliers management page
│   ├── users.html                # User management page
│   └── activityLog.html          # Activity log page
│
├── js/
│   ├── main.js                   # Entry point — routes by current page
│   ├── jquery-3.7.1.js           # jQuery library
│   ├── components/
│   │   ├── modal.js              # Reusable modal component
│   │   └── navbar.js             # Navbar component
│   └── pages/
│       ├── dashboard.js          # Dashboard logic & quick actions
│       ├── login.js              # Login page controller
│       ├── register.js           # Registration page controller
│       ├── users.js              # Users tab controller
│       ├── orders.js             # Orders page controller
│       ├── activityLog.js        # Activity log tab controller
│       ├── products.js           # Products tab controller
│       └── reports.js            # Reports tab controller
│
├── services/                     # API communication layer
│   ├── fetch.js                  # Shared fetch helpers & product/category
data
│   ├── authService.js            # Authentication (login, logout, session)
│   ├── userService.js            # User CRUD operations
│   ├── productService.js         # Product CRUD & stock adjustment logic
│   ├── orderService.js           # Order CRUD & receive flow
│   ├── supplierService.js        # Supplier CRUD logic (jQuery-based)
│   ├── reportService.js          # Report data fetching & rendering
│   ├── activityLogService.js     # Activity log CRUD operations
│   ├── categoryService.js        # Category operations
│   └── inventoryService.js       # Inventory helpers
│
├── models/                       # Data models with validation
```

```
│   ├── User.js                  # User model (name, email, password, role)
│   ├── Product.js               # Product model
│   ├── Order.js                 # Order model (id, supplierId, productId,
qty, status, date)
│   ├── Supplier.js              # Supplier model (SSC, name, phone, email,
address)
│   └── ActivityLog.js           # Activity model (action, entity, userId,
date)
│
├── components/
│   ├── modal.html               # Modal HTML template
│   └── navbar.html              # Navbar HTML template
│
├── assets/
│   ├── css/
│   │   ├── main.css
│   │   ├── modal.css
│   │   ├── navbar.css
│   │   ├── orders.css
│   │   ├── product.css
│   │   ├── report.css
│   │   ├── supplier.css
│   │   └── table.css
│   └── Images/
│       ├── analytics.png
│       ├── login-navbar-icon.png
│       ├── navbaricon.png
│       ├── overlay_register.png
│       ├── sync.png
│       └── warehouse.png
│
├── db/
│   ├── products.json            # JSON Server database file
│   └── users.json               # Users data file
│
└── utils/
    ├── helpers.js               # Shared utility helpers
    ├── validation.js            # Shared validation utilities
    └── Error handlers/
        ├── BaseError.js
        ├── FileTypeError.js
        ├── InternalError.js
        ├── InvalidCredentialError.js
        ├── NotFoundError.js
        ├── ResourceAlreadyExistError.js
        ├── RouteNotFoundError.js
        ├── UnauthorizedError.js
        └── ValidationError.js
```

---

3. API Endpoints

The application uses **JSON Server** running at `http://localhost:3000`. All endpoints follow REST conventions.

3.1 Users — `/users`

Method	Endpoint	Description
GET	`/users`	Get all users
GET	`/users?email={email}`	Find user by email (used for login & duplicate check)
GET	`/users/{id}`	Get a specific user by ID
POST	`/users`	Create a new user
PUT	`/users/{id}`	Fully update a user
DELETE	`/users/{id}`	Delete a user

User object fields: `id`, `name`, `email`, `password`, `role` (`"admin"` or `"user"`)

3.2 Products — `/product`

Method	Endpoint	Description
GET	`/product`	Get all products
GET	`/product/{id}`	Get a specific product by ID
POST	`/product`	Create a new product
PATCH	`/product/{id}`	Partially update a product (e.g., update stock)
DELETE	`/product/{id}`	Delete a product

Product object fields: `id`, `name`, `category`, `supplier`, `stock`, `Reorder_Level`, `sku`, `image`, `price`

3.3 Suppliers — `/suppliers`

Method	Endpoint	Description
GET	`/suppliers`	Get all suppliers
GET	`/suppliers/{id}`	Get a specific supplier by ID
POST	`/suppliers`	Create a new supplier
PUT	`/suppliers/{id}`	Fully update a supplier
DELETE	`/suppliers/{id}`	Delete a supplier

Supplier object fields: `id`, `ssc`, `name`, `phone`, `email`, `address` (`continent`, `country`, `city`, `street`)

3.4 Orders — `/orders`

Method	Endpoint	Description
GET	`/orders`	Get all orders
GET	`/orders/{id}`	Get a specific order by ID
POST	`/orders`	Create a new purchase order
PUT	`/orders/{id}`	Update an order (only if status is `"pending"`)
DELETE	`/orders/{id}`	Delete an order (only if status is `"pending"`)

Order object fields: `id`, `supplierId`, `supplierName`, `productId`, `productName`, `sku`, `quantity`, `status` (`"pending"` / `"received"`), `date`

> **Business rule:** Orders with a status of `"received"` cannot be edited or deleted.

3.5 Activity Logs — `/activityLogs`

Method	Endpoint	Description
GET	`/activityLogs`	Get all activity log entries
GET	`/activityLogs/{id}`	Get a specific log entry
POST	`/activityLogs`	Create a new log entry
DELETE	`/activityLogs/{id}`	Delete a log entry

Activity object fields: `id`, `action`, `entity` (one of: `products`, `categories`, `orders`, `suppliers`, `users`), `userId`, `date`

3.6 Categories — `/categories`

Method	Endpoint	Description
GET	`/categories`	Get all categories
POST	`/categories`	Create a new category
DELETE	`/categories/{id}`	Delete a category

Category object fields: `id`, `name`, `description`

3.7 Stock Adjustments — `/adjustment`

Method	Endpoint	Description
GET	`/adjustment`	Get all adjustment records
POST	`/adjustment`	Record a new stock adjustment

Adjustment object fields: `id`, `product_Name`, `type` (`"increase"` / `"decrease"`), `quantity`, `reason`, `old_Quantity`, `New_Quantity`

---

4. Pages & Features

4.1 Login Page (`login.html`)

The entry point for all users. Unauthenticated visitors are directed here.

Features: - Email and password input fields - Credentials validated against the `/users` API endpoint (query by email) - On success, the authenticated user object is stored in `localStorage` under the key `currentUser` - On failure, an `InvalidCredentialError` is thrown and an inline error message is shown - Redirect to `index.html` (main dashboard) upon successful login - Link to the registration page for new users

---

4.2 Registration Page (`register.html`)

Allows new users to create an account.

Features: - Input fields: Full Name, Email, Password, Role - Client-side validation using the `User` model: - Name: letters only, minimum 3 characters - Email: valid email format required - Password: minimum 6 characters, must include at least one capital letter and one number - Duplicate email check — returns a `ResourceAlreadyExistError` if the email already exists - On success, the user is created and redirected to login

---

4.3 Dashboard (`index.html`)

The main application shell. The sidebar navigation loads content dynamically into a central content area without full page reloads (except for Orders and Suppliers pages).

Features:

Summary Cards: - Total Products count - Total Suppliers count - Low Stock Items count (products where `stock < Reorder_Level`) - Total Inventory Units (sum of all stock quantities)

Low Stock Table: - Lists all products where current stock is below their reorder level - Color-coded: red for stock ≤ 10 units, orange for stock between 10 and 2x reorder level

Recent Activity Feed: - Shows the 5 most recent activity log entries - Displays action type, entity affected, user name, and date

Quick Action Buttons: - **Add Product** — opens a modal to create a new product with all fields including category dropdown - **Add Category** — opens a modal to create or delete categories; category list updates live - **Adjust Stock** — opens a modal to increase or decrease stock for any product by SKU; records an adjustment entry in the database

Sidebar Navigation: - Dashboard (current view) - Users (loads user management panel — admin only) - Products (loads product management panel) - Suppliers (navigates to `pages/suppliers.html`) - Orders (navigates to `pages/orders.html`) - Reports (navigates to `pages/reports.html`) - Activity Log (loads activity log panel)

Role-Based Access: - The "Users" sidebar item is hidden for non-admin roles - Role is checked on page load from the `currentUser` localStorage session

---

4.4 Products Page (`pages/products.html`)

Manages the complete product catalog.

Features: - **Product table** — displays image, name, category, supplier, stock quantity with color status indicator, reorder level, and SKU - **Stock status indicator** — a colored dot per product: green (safe), orange (approaching reorder level), red (below reorder level) - **Add Product form** — toggleable form with fields: name, category (dropdown from API), supplier, quantity, reorder level, SKU, and image URL - **Edit Product form** — pre-fills with current product data; allows updating name, category, supplier, reorder level, SKU, and image via PATCH request - **Delete Product** — removes a product from the database with confirmation - **Search** — real-time filtering across name, category, supplier, SKU, and stock fields - **Stock Adjustment form** — selects product by SKU, shows current stock, chooses increase or decrease action, enters new target value, records the change in `/adjustment`

4.5 Suppliers Page (`pages/suppliers.html`)

Full supplier management.

Features: - **Supplier table** — displays SSN, name, phone, email, continent, country, city, and street address - **Add Supplier form** — fields: SSN, Name, Phone, Email, Continent, Country, City, Street; toggles visibility of the main table - **Client-side validation:** - All fields required - Email must be a valid format - Name must be at least 5 characters - Phone must be exactly 11 digits - **Update Supplier** — loads the existing supplier data into the add form; submits via PUT - **Delete Supplier** — triggers a Bootstrap confirmation modal before permanent deletion - **Search** — real-time filtering by supplier name

---

4.6 Orders Page (`pages/orders.html`)

Manages purchase orders from creation through fulfillment.

Features: - **Orders table** — displays order ID, supplier name (with ID), product name (with SKU), quantity, status badge (color-coded), and date - **Create New PO button** — opens a modal for entering Supplier ID, Product ID, and Quantity; auto-assigns ID, status `"pending"`, and current date - **Edit Order** — available only for `"pending"` orders; pre-fills the modal with existing order data - **Receive Order** — marks order status as `"received"`

via PUT; triggers a confirmation prompt; button hidden for non-pending orders - **Delete Order** — available only for `"pending"` orders; triggers a confirmation prompt - **Filters:** - Text search by product name, order ID, or supplier name - Status dropdown (All / pending / received) - Supplier dropdown (dynamically populated from order data) - **Activity Logging** — every create, update, receive, or delete action automatically logs an entry to `/activityLogs` - **Sidebar navigation** — clicking any item other than Orders redirects back to `index.html`

4.7 Reports Page (`pages/reports.html`)

Provides inventory analytics and reporting.

Features:

Summary Metrics Cards: - Total Inventory Value (displayed as currency) - Total Products count - Low Stock Count (with "Requires Attention" warning badge)

Low Stock Report table: - Lists all products where `stock < Reorder_Level` - Columns: Product Name, Current Quantity (highlighted in red), Reorder Level, Supplier, Action - "Order Now" button per row (links to ordering flow) - "Reorder All" shortcut link at the top

Inventory Value Report table: - Lists all products with their unit price and total value (stock × price) - Columns: Product Name, Quantity On Hand, Unit Price, Total Value - Running total of overall inventory value displayed at the bottom

Export: - "Export PDF" button available in the page header

Search: - Search input to filter report content

4.8 Users Page (loaded inside dashboard)

Available to admin users only (hidden in sidebar for regular users).

Features: - **Summary cards** — Total Users count and Total Admins count - **Users table** — columns: Name, Email, Password, Role (badge), Actions - **Add User button** — opens a modal/form to create a new user with validation - **Edit User (pencil icon)** — opens pre-filled form to update user details - **Delete User (trash icon)** — removes user with confirmation - **Role badge** — blue badge for Admin, grey badge for User

---

4.9 Activity Log Page (loaded inside dashboard)

A full audit trail of all system actions.

Features: - **Activity table** — columns: Action, Entity, User (resolved from user ID to name), Date, Delete - **Delete Log Entry (trash icon)** — removes a specific log entry with confirmation - **Search** — real-time filtering across all log fields (action, entity, user, date)

---

5. Services

Service	File	Responsibilities
`AuthService`	`services/authService.js`	`login()`, `logout()`, `register()`, `getCurrentUser()`, `isAuth()`, `isAdmin()` — manages localStorage session
`UserService`	`services/userService.js`	`createUser()`, `getAllUsers()`, `getUserById()`, `updateUser()`, `deleteUser()`
`OrderService`	`services/orderService.js`	`getall()`, `getbyId()`, `create()`, `update()`, `delete()`, `receive()` — enforces pending-only edit/delete rule
`ActivityService`	`services/activityLogService.js`	`createActivity()`, `getAllActivities()`, `getActivityById()`, `deleteActivity()`
`productService`	`services/productService.js`	`initProductPage()` — handles full product page lifecycle including render, search, add, edit, delete, and stock adjustment
`supplierService`	`services/supplierService.js`	jQuery-based service handling full supplier page lifecycle including render, search, add, update, delete with modal confirmation
`reportService`	`services/reportService.js`	Fetches products and renders the low-stock table and inventory value table
`fetch.js`	`services/fetch.js`	Shared module that pre-fetches product and category data at load time; exports `add_Product_API` helper

---

6. Models & Validation

User (`models/User.js`)

- **name**: Required; letters only (no digits/special chars); minimum 3 characters
- **email**: Required; must match valid email format
- **password**: Required; minimum 6 characters; must contain at least one uppercase letter and one digit
- **role**: Defaults to `"user"`; can be `"admin"`

Order (`models/Order.js`)

- Constructed with: `id`, `supplierId`, `productId`, `quantity`
- **status**: Defaults to `"pending"` if not provided
- **date**: Defaults to today's ISO date string if not provided

Supplier (`models/Supplier.js`)

- Fields: `ssc`, `name`, `phone`, `email`, `continent`, `country`, `city`, `street`
- All fields exposed via private class fields with getters and setters

Activity Log (`models/ActivityLog.js`)

- **action**: Required; minimum 3 characters
- **entity**: Required; must be one of: `products`, `categories`, `orders`, `suppliers`, `users`
- **userId**: Required
- **date**: Required

---

7. Error Handling

All error classes extend `BaseError` located in `utils/Error handlers/`:

Class	When Thrown
`ValidationError`	Invalid input data (name format, password rules, etc.)
`InvalidCredentialError`	Email not found or password mismatch during login
`ResourceAlreadyExistError`	Duplicate email on registration or user creation
`NotFoundError`	Requested resource (user, activity) does not exist (404)
`InternalError`	Unexpected server or fetch error
`UnauthorizedError`	Access attempt without authentication
`RouteNotFoundError`	Navigation to an undefined route
`FileTypeError`	Invalid file type provided

All service methods wrap their API calls in `try/catch` and re-throw known errors while wrapping unknown errors in `InternalError`.

---

8. How to Run the Project

Prerequisites

Make sure the following are installed on your machine:

- **Node.js** (v14 or higher) — https://nodejs.org
- **npm** (comes bundled with Node.js)
- A modern web browser (Chrome, Firefox, Edge)
- A local static file server (e.g., VS Code **Live Server** extension, or `npx serve`)

Step 1 — Install JSON Server

JSON Server acts as the backend REST API. Install it globally:

```
npm install -g json-server
```

Step 2 — Start the JSON Server

Navigate to the project root directory and start the server pointing to the database file:

```
cd "Inventory System ITI/Inventory-System"
json-server --watch db/products.json --port 3000
```

The API will be available at: `http://localhost:3000`

You should see the following routes listed in the terminal:

```
Resources:
  http://localhost:3000/users
  http://localhost:3000/product
  http://localhost:3000/suppliers
  http://localhost:3000/orders
  http://localhost:3000/activityLogs
  http://localhost:3000/categories
  http://localhost:3000/adjustment
```

Step 3 — Serve the Front-End

Open a second terminal in the project root. You can use any of the following options:

Option A — VS Code Live Server (recommended): - Open the project folder in VS Code - Right-click `login.html` → **"Open with Live Server"**

Option B — npx serve:

```
npx serve .
```

Then open `http://localhost:3000` (or the port shown) in your browser.

Option C — Python HTTP Server:

```
python -m http.server 5500
```

Then open `http://localhost:5500/login.html`

> **Important:** The front-end must be served through a local HTTP server — not opened directly as a `file://` URL — because ES Modules (`import/export`) are blocked by browsers for local files.

Step 4 — Open the Application

Navigate to the login page in your browser:

`http://localhost:5500/login.html`

Use an existing account from `db/users.json`, or create a new one via the Register page.

Default Credentials (from `db/users.json`)

Check `db/users.json` for pre-seeded accounts. Admin accounts have `"role": "admin"` and have access to the Users management section.

Configuration Notes

- All API base URLs are hardcoded to `http://localhost:3000` in the service files. If you run JSON Server on a different port, update the `baseUrl` constant in each service file accordingly.
- The application stores the current user session in `localStorage`. Closing the browser tab does **not** log the user out. To log out, click the logout icon in the sidebar or call `AuthService.logout()` from the browser console.
- Data persists in `db/products.json` — changes made through the app are written back to this file by JSON Server automatically.